

ia-26/DE.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import functions as f
5
6 class Individual:
7
8     def __init__(
9         self,
10         dim,
11         bounds,
12         objective_function
13     ):
14
15         self.objective_function = objective_function
16
17         self.vector = np.random.uniform(
18             bounds[0],
19             bounds[1],
20             dim
21         )
22
23         self.fitness = self.objective_function(
24             self.vector
25         )
26
27     def evaluate(self):
28
29         self.fitness = self.objective_function(
30             self.vector
31         )
32
33
34 def differential_evolution(
35     objective_function,
36     dim=30,
37     population_size=100,
38     generations=1000,
39     bounds=(-100, 100),
40     F=0.8,
41     CR=0.9
42 ):
43
44     # Inicializa população
45     population = [
46
47         Individual(
48             dim,
49             bounds,
50             objective_function
```

```
51     )
52
53     for _ in range(population_size)
54 ]
55
56 # Melhor solução global
57 best_individual = min(
58     population,
59     key=lambda ind: ind.fitness
60 )
61
62 history = []
63
64 # Loop principal
65 for generation in range(generations):
66
67     elite = min(
68         population,
69         key=lambda ind: ind.fitness
70     )
71
72     for i in range(population_size):
73
74         target = population[i]
75
76         # Escolhe 3 indivíduos distintos
77         indices = list(range(population_size))
78
79         indices.remove(i)
80
81         a_idx, b_idx, c_idx = np.random.choice(
82             indices,
83             3,
84             replace=False
85         )
86
87         a = population[a_idx]
88         b = population[b_idx]
89         c = population[c_idx]
90
91         # Mutação
92         #  $v = a + F \cdot (b - c)$ 
93         mutant_vector = (
94             a.vector
95             + F * (b.vector - c.vector)
96         )
97
98         mutant_vector = np.clip(
99             mutant_vector,
100             bounds[0],
101             bounds[1]
102         )
103
```

```

104     # Crossover binomial
105     trial_vector = np.copy(target.vector)
106
107     j_rand = np.random.randint(dim)
108
109     for j in range(dim):
110
111         if (
112             np.random.rand() < CR
113             or j == j_rand
114         ):
115
116             trial_vector[j] = mutant_vector[j]
117
118     # Avalia vetor teste
119     trial_fitness = objective_function(
120         trial_vector
121     )
122
123     # Seleção
124     if trial_fitness < target.fitness:
125
126         target.vector = trial_vector
127         target.fitness = trial_fitness
128
129     #ELITISMO
130     worst = max(
131         population,
132         key=lambda ind: ind.fitness
133     )
134
135     population.remove(worst)
136     population.append(elite)
137
138     # Melhor indivíduo da geração
139     current_best = min(
140         population,
141         key=lambda ind: ind.fitness
142     )
143
144     if current_best.fitness < best_individual.fitness:
145
146         best_individual = current_best
147
148     history.append(best_individual.fitness)
149
150     print(
151         f"Geração {generation+1:4d}"
152         f" | Melhor valor: "
153         f"{best_individual.fitness:.10f}"
154     )
155
156     return (

```

```

157         best_individual.vector,
158         best_individual.fitness,
159         history
160     )
161
162     def plot_convergence(history):
163
164         plt.figure(figsize=(8, 5))
165
166         plt.plot(history)
167
168         plt.title("Convergência do Differential Evolution")
169
170         plt.xlabel("Geração")
171         plt.ylabel("Melhor valor")
172
173         plt.yscale("log")
174
175         plt.grid(True)
176
177         plt.show()
178
179
180     if __name__ == "__main__":
181
182         objective_function = f.f1
183
184         bounds = (-100, 100)
185
186         best_position, best_value, history = (
187             differential_evolution(
188                 objective_function=objective_function,
189                 dim=30,
190                 population_size=300,
191                 generations=350,
192                 bounds=bounds,
193                 F=0.5,
194                 CR=0.7
195             )
196         )
197
198         plot_convergence(history)
199
200         print("\n===== RESULTADO FINAL =====")
201
202         print("Melhor posição:")
203         print(best_position)
204
205         print("\nMelhor valor:")
206         print(best_value)

```